

MealBridge Deliverable 2 and 3: Innovation Proposal and Software Requirements

Course	BCSE302P Database Systems Lab, Societal Digital Innovation Project
Track	T5 Waste and Circular Economy (surplus resources)
Status	Draft v1, 5 October 2026
Reads with	Deliverable 1 (01_problem_discovery.md) and Deliverable 4 (03_database_design.md)

How to read this document. Section 1 is the innovation argument: what is new and why it needs a database. Sections 2 to 6 are the software requirements. Every requirement has an ID (FR-n, NFR-n) and a column saying where it is implemented and how it is tested, so the internal review can be checked line by line. Numbers that fieldwork must supply are marked **TO COLLECT**; unsourced planning values are marked **TO VERIFY**, exactly as in Deliverable 1.

1. Innovation proposal

1.1 The gap, in one paragraph

Surplus cooked food in a hostel mess has a few hours of safe life. In that window someone must notice it, find a recipient who can actually use *that* food (diet, quantity, storage) and reach it in time, arrange transport, and leave a record that it was handled safely. Today this runs on phone calls and WhatsApp groups. Existing platforms do not close this gap: Feeding India runs meal programmes rather than surplus rescue, Robin Hood Army's own platform logs volunteer drives rather than matching donors to recipients, OLIO and Too Good To Go are not operating in India and are built for individuals buying or collecting small lots, and FSSAI's IFSA is a directory. No Food Waste (Coimbatore) is the closest Indian model and still publishes nothing about expiry deadlines, fairness between recipients or concurrency-safe claiming. Deliverable 1, section 4 has the verified comparison table.

1.2 What MealBridge does differently

MealBridge is a **database-enforced matching layer** for perishable institutional surplus. The claim is deliberately narrow: the rules that matter are enforced by the schema and by transactions, not by the goodwill of whoever is in the group chat.

#	Innovation	Why it needs the database	Where it lives
I1	Perishability clock. Each batch gets a safe-until deadline from its food category, storage mode and cooking time. Food already past its deadline cannot be posted, and the deadline cannot be edited afterwards.	A trigger computes and freezes the deadline, so no client (app, script or psql session) can set a convenient expiry. The deadline is a snapshot, so later edits to the reference table never change already-posted food.	<code>fn_safe_until</code> , <code>trg_batch_bi</code> , <code>trg_batch_bu</code>

#	Innovation	Why it needs the database	Where it lives
12	Fair match score. Shelters are ranked on five components: unmet need today, fairness over the last 30 days, distance, time safety margin and best capacity fit. Hard filters remove any shelter that excludes the batch's diet tags, lacks capacity for the whole batch, is beyond the service radius, or cannot be reached before the deadline with a 30 minute buffer.	The score is computed in SQL from live data (need, reserved capacity, past receipts), and the weights live in a table so policy changes need no code change. Every weight change is audited.	<code>fn_match_score</code> and its five component functions, <code>sp_rank_shelters</code> , <code>scoring_weight</code> , <code>trg_weight_au</code>
13	Concurrency-safe claiming. Two shelters can never hold the same batch.	Three independent layers: a row lock (<code>SELECT ... FOR UPDATE</code>) inside the claim transaction, a trigger that refuses a claim on a batch that is not AVAILABLE, and a UNIQUE index on a generated column that allows at most one live claim per batch.	<code>sp_claim_batch</code> , <code>trg_claim_bi</code> , <code>claim.active_batch_id</code>
14	Capacity that cannot be exceeded. A shelter cannot be sent more than it can take that day.	A trigger keeps <code>shelter_day.reserved_kg</code> and a CHECK constraint refuses any claim that would push it past <code>capacity_kg</code> ; the failing CHECK undoes the whole claim.	<code>trg_claim_ai</code> , <code>trg_claim_au</code> , <code>chk_sd_reserved</code>
15	Volunteer pickup batching. One trip covers several messes and several shelters, with per-stop ETAs, and the trip is refused if any batch would arrive after its deadline.	Stop order and ETAs are computed with window functions (<code>LAG</code> , <code>SUM()</code> OVER) over spatial distances; the perishability check is part of the same transaction as the trip.	<code>sp_create_trip</code>
16	Tamper-evident chain of custody. Every hand-over (cooked, packed, posted, claimed, picked up, hygiene-checked, delivered) is an append-only row whose hash includes the previous row's hash.	UPDATE and DELETE are refused by triggers and not granted to any role, including the platform admin; a verifier function recomputes the chain and names the first altered row.	<code>custody_event</code> , <code>trg_custody_bi/bu/bd</code> , <code>fn_custody_first_bad_event</code>
17	Surplus forecasting and pre-alerts. Shelters are warned before the surplus exists, from the same-weekday average of the last four weeks of mess attendance and leftovers.	The forecast is a SQL procedure over the mess meal log, and its output is stored so its accuracy can be measured against what actually happened.	<code>sp_generate_forecast</code> , <code>surplus_forecast</code> , <code>mess_meal_log</code>
18	Impact and fairness reporting. Meals saved, kg diverted, carbon avoided, response time and a fairness index, each with one SQL definition.	Views, so every number on a dashboard can be traced to one SELECT and no figure is computed twice in two places.	<code>v_impact_summary</code> , <code>v_impact_daily</code> , <code>v_response_time</code> , <code>v_shelter_fairness</code> , <code>v_fairness_index</code> , <code>v_mess_leaderboard</code>

#	Innovation	Why it needs the database	Where it lives
I9	The database made visible. The app shows the SQL, transaction and triggers behind each action, a live two-session race, query plans with and without each index, the custody timeline and a schema explorer.	Nothing extra is needed in the database for this; the API records the statements it really sent and reads <code>information_schema</code> and <code>performance_schema</code> .	<code>app/backend/mealbridge_api/db.py</code> , the four pages under <code>app/frontend/app/lab/</code>

1.3 What is deliberately *not* claimed

- The forecast is a four-week same-weekday average. On the synthetic data its mean absolute error is roughly 35 to 55 per cent. It is a usable pre-alert, not an accurate prediction, and the document says so wherever it appears.
- Distance is straight-line (great-circle) multiplied by a detour factor. A routing API would replace it; the trip order is a simple, explainable heuristic, not an optimal route.
- Safe-hours per food category, `kg_per_meal` and the carbon factor are planning values marked **TO VERIFY** in `food_category.values_source`. No impact figure in this project is evidence about real messes until fieldwork and a cited factor replace them.
- There is no machine learning in the system. Nothing in the problem needs it, and a four-week average is defensible in a viva in a way a black box is not.

1.4 Societal impact, and how it will be measured

The honest position before fieldwork: the impact figures the system reports are **calculations over its own records**, so they become evidence only once real messes use it. What the design does guarantee is that the records needed to measure impact exist.

Outcome	Metric	Where it comes from	Baseline
Food diverted from waste	kg delivered per mess per week	<code>v_impact_daily</code>	TO COLLECT in the mess interviews (Deliverable 1, section 5.1 Q3 to Q5)
Meals provided	meal-equivalents (<code>fn_meals</code>)	<code>v_impact_summary</code>	TO COLLECT from shelters' current shortfall
Speed	minutes from posting to first claim, and to delivery	<code>v_batch_outcome</code> , <code>v_response_time</code>	current practice: TO COLLECT (how long a WhatsApp handover takes today)
Fairness	Jain's index over kg per beneficiary, and each shelter's fair ratio	<code>v_fairness_index</code> , <code>v_shelter_fairness</code>	no baseline exists today, which is itself the finding
Emissions avoided	kg CO ₂ e	<code>v_batch_outcome</code> × <code>food_category.co2e_kg_per_kg</code>	factor TO VERIFY (currently one global FAO 2013 average)
Safety and compliance	share of batches with a complete custody chain and a passed hygiene check	<code>custody_event</code> , <code>fn_custody_first_bad_event</code>	current practice keeps no record at all

1.5 Maturity (TRL)

Target TRL 4 to 5. The evidence for it: the database runs on MySQL 8.0.46 with 24 tables, 12 functions, 12 procedures, 18 triggers, 10 views and role-based access, loaded with about 30 days of synthetic history; the concurrency guarantee, the index effects and the role grants are demonstrated with real

captured output (`sql/*.output.md` , 66 of 66 access tests passing); and the web application drives the same procedures end to end. What is still missing for TRL 5 is a field trial with one real mess and one real shelter, which is the next step after the internal review.

2. Stakeholders and what each needs

Stakeholder	Goal	What the system must give them	Role in the system
Mess manager or contractor	Give away surplus in under a minute, without liability worry	One short form; automatic deadline; a record of where the food went	MESS_ADMIN (MySQL role <code>r_mess_admin</code>)
Mess kitchen staff	Not more work at the busiest time	The same form, on a phone; no reporting duties	acts through the mess admin account
Shelter staff	Enough suitable food, early enough to use it	A feed ranked for them, a one-click claim, and pre-alerts	SHELTER (<code>r_shelter</code>)
Volunteer	A route that is worth the trip	Batched multi-stop trips with ETAs and contact numbers	VOLUNTEER (<code>r_volunteer</code>)
Platform admin (student team, later an NGO)	Keep matching fair and auditable	Policy weights, the audit log, the jobs	PLATFORM_ADMIN (<code>r_platform_admin</code>)
Institution and estate office	Less waste, and evidence of it	Reports per mess and per month	reads the impact dashboard
Food safety authority (FSSAI framework)	Traceability of donated food	The custody chain per batch, on demand	not a user; the record is the deliverable
Hostel students	Their food not wasted; a way to help	The public impact page; volunteering	public pages, or a volunteer account

3. Functional requirements

Priority: **M** must have for the internal review, **S** should have, **C** could have later.

3.1 Accounts and access

ID	Requirement	Pri	Implemented in	Tested by
FR-1	A user logs in with e-mail and password; passwords are stored as bcrypt hashes, never plain text.	M	<code>app_user.password_hash</code> , <code>auth.py</code>	<code>test_api.py::test_wrong_password_is_refused</code>
FR-2	Each user has exactly one of four roles, and a mess or shelter user belongs to exactly one site of the matching type.	M	<code>chk_user_site_by_role</code> , <code>trg_user_bi</code> , <code>trg_user_bu</code>	<code>02_constraint_smoke_test.sql</code>

ID	Requirement	Pri	Implemented in	Tested by
FR-3	The API connects to MySQL with one database login per role, so MySQL's grants, not application code, decide what is reachable.	M	<code>config.DB_LOGINS</code> , <code>08_roles_grants.sql</code>	<code>rbac_test.py</code> (66 tests), <code>test_api.py::test_hood_reports_the_role_login</code>
FR-4	Every end-user write goes through a SQL SECURITY DEFINER procedure that re-checks role and ownership; no role has direct INSERT on <code>surplus_batch</code> or <code>claim</code> .	M	<code>04_procedures.sql</code> , grants	<code>rbac_test.py</code> , <code>test_api.py::test_mess_cannot_cancel_other_mess_batch</code>
FR-5	A user may read and change only their own site's rows.	M	<code>p_user_id</code> checks inside each procedure	<code>rbac_test.py</code> REFUSED rows
FR-6	A platform admin can register a new mess or shelter, with the site row and its subtype row created together or not at all.	S	<code>sp_register_site</code>	<code>09_demo...output.md</code> section P3

3.2 Posting surplus

ID	Requirement	Pri	Implemented in	Tested by
FR-7	A mess admin posts a batch with food category, meal slot, description, quantity, storage mode, cooking time and diet tags.	M	<code>sp_post_batch</code>	<code>test_api.py</code> flow

ID	Requirement	Pri	Implemented in	Tested by
FR-8	The system, not the client, sets the safe-until deadline from category, storage and cooking time.	M	trg_batch_bi with fn_safe_until	test_api.py::test_trigger_sets_the_perishability_deadline
FR-9	Food already past its deadline, or cooked in the future, cannot be posted.	M	trg_batch_bi, chk_batch_order	02_constraint_smoke_test.sql
FR-10	The deadline inputs cannot be edited after posting; the quantity cannot change once claimed.	M	trg_batch_bu	09_demo...output.md section T3
FR-11	Posting a batch alerts the staff of the best-matching shelters.	S	sp_post_batch, notification	09_demo...output.md section P1
FR-12	A mess admin can withdraw a batch while it is still unclaimed.	M	sp_cancel_batch	test_api.py
FR-13	A mess admin sees each batch's remaining safe time, its status, and who claimed it.	M	/api/mess/overview, countdown ring	screenshot, mess dashboard

3.3 Matching and claiming

ID	Requirement	Pri	Implemented in	Tested by
FR-14	A shelter sees only batches that are available and not past their deadline.	M	v_live_feed	test_api.py::test_shelter_feed_ranks_and_claims
FR-15	The feed is ranked by a match score computed from need, fairness, distance, time margin and capacity fit.	M	fn_match_score	09_demo...output.md section P1

ID	Requirement	Pri	Implemented in	Tested by
FR-16	A shelter that excludes a diet tag of the batch, has no capacity for the whole batch, is beyond the radius, or cannot be reached in time, is excluded with a stated reason.	M	hard filters in <code>fn_match_score</code> , <code>sp_rank_shelters</code>	ranking table in the UI; <code>rbac_test.py</code>
FR-17	A batch is claimed whole; partial claims do not exist.	M	schema design (no quantity on <code>claim</code>)	design doc 4.3
FR-18	Two shelters claiming the same batch at the same time results in exactly one claim; the other is told why.	M	<code>sp_claim_batch</code> , <code>trg_claim_bi</code> , UNIQUE on <code>claim.active_batch_id</code>	<code>13_race_demo.output.txt</code> , <code>test_api.py::test_race_has_one_winner_one_loser</code>
FR-19	A claim immediately reserves the shelter's capacity for that day and can never exceed it.	M	<code>trg_claim_ai</code> , <code>chk_sd_reserved</code>	<code>test_api.py::test_capacity_check_constraint</code>
FR-20	A shelter can cancel a claim before pickup; the capacity is released and the food is re-offered if still safe.	M	<code>sp_cancel_claim</code> , <code>trg_claim_au</code>	<code>10_dml_examples.output.md</code>
FR-21	A shelter can set its own meals needed and capacity for the day, but not the reserved counter.	M	column-level grants on <code>shelter_day</code>	<code>rbac_test.py</code>
FR-22	The matching policy weights can be changed by a platform admin without a code change, and every change is audited.	S	<code>scoring_weight</code> , <code>trg_weight_au</code>	<code>09_demo...output.md</code> section P2

3.4 Logistics

ID	Requirement	Pri	Implemented in	Tested by
FR-23	A volunteer sees claimed batches waiting for pickup, without being able to read match scores or shelter capacity.	M	<code>v_pickup_queue</code> plus grants	<code>rbac_test.py</code>

ID	Requirement	Pri	Implemented in	Tested by
FR-24	A volunteer can plan one trip covering several claims, several messes and several shelters.	M	sp_create_trip	test_api.py::test_volunteer_trip_pickup_delivery
FR-25	A trip is refused if its total load exceeds the volunteer's vehicle limit, if a claim is already on another trip, or if any batch would arrive after its deadline.	M	sp_create_trip	09_demo...output.md, test_api.py
FR-26	Each stop gets a planned arrival time computed from the route.	M	sp_create_trip (window functions)	trip timeline in the UI
FR-27	Only an available, ID-verified volunteer can take a trip.	M	sp_create_trip	rbac_test.py
FR-28	A volunteer records each pickup with the food temperature, and each delivery with a hygiene check result.	M	sp_record_pickup, sp_record_delivery	test_api.py
FR-29	A failed hygiene check rejects the claim instead of marking it delivered, and releases the capacity.	M	sp_record_delivery, trg_claim_au	09_demo...output.md section T8
FR-30	A volunteer can go on and off duty.	S	sp_set_availability	rbac_test.py

3.5 Safety, expiry and audit

ID	Requirement	Pri	Implemented in	Tested by
FR-31	Every stage of a batch's life is recorded as an append-only custody event with a hash of the previous event.	M	custody_event, trg_custody_bi	test_api.py::test_custody_chain_is_intact
FR-32	Custody events can never be updated or deleted, by any role, including the database root user.	M	trg_custody_bu, trg_custody_bd, grants	09_demo...output.md section T12, rbac_test.py
FR-33	Any alteration of the custody history can be detected and the first altered row named.	M	fn_custody_first_bad_event	09_demo...output.md section T13

ID	Requirement	Pri	Implemented in	Tested by
FR-34	Batches past their deadline are expired automatically, and live claims on expired food are cancelled.	M	<code>sp_expire_batches</code> , <code>event ev_auto_expire</code> every 5 minutes	<code>09_demo...output.md</code> section T10
FR-35	Between two runs of the expiry job, expired food is still never offered or claimable.	M	<code>v_live_feed</code> filter, <code>trg_claim_bi</code>	<code>02_constraint_smoke_test.sql</code>
FR-36	Changes to batches, claims, policy weights and food-safety reference values are written to an audit log with the real database login that made them.	M	audit triggers using <code>USER()</code>	admin page, <code>09_demo...output.md</code>

3.6 Forecasting, reports and transparency

ID	Requirement	Pri	Implemented in	Tested by
FR-37	The system forecasts tomorrow's surplus per mess and meal slot from the last four same weekdays, stores it, and pre-alerts shelters above a threshold.	S	<code>sp_generate_forecast</code>	<code>11_queries.output.md</code> Q17
FR-38	Forecast accuracy can be measured later against what actually happened.	S	<code>surplus_forecast</code> kept, compared with <code>mess_meal_log</code>	<code>11_queries.output.md</code> Q17
FR-39	The impact dashboard reports meals saved, kg diverted, carbon avoided, rescue rate, response time, fairness and a mess leaderboard, each from one view.	M	<code>06_views.sql</code> , <code>/api/impact</code>	<code>test_api.py::test_impact_views_follow_grants</code>
FR-40	A view a role is not granted is shown as locked with MySQL's own error, not hidden.	S	<code>/api/impact</code> per-view error capture	<code>test_api.py::test_impact_views_follow_grants</code>
FR-41	The public landing page shows live aggregate impact without a login, using a service login that can read nothing else.	S	<code>r_public</code> , <code>/api/public/impact</code>	<code>rbac_test.py</code> (public role tests)

ID	Requirement	Pri	Implemented in	Tested by
FR-42	For each action, the application can show the SQL that ran, the transaction, the triggers that fired and the rows those triggers wrote.	M	db.py recorder, hood.py, the Under the hood panel	every screenshot in the README
FR-43	The application can show the live two-session race, including the row locks InnoDB held.	M	/api/lab/race	test_api.py::test_race_has_one_winner_one_loser
FR-44	The application can show query plans with and without each index, with real row counts and milliseconds.	M	/api/lab/explain	test_api.py::test_explain_before_and_after
FR-45	The application can show the schema (tables, columns, keys, CHECK constraints, indexes), the ER diagrams and the source of every procedure, function, trigger and view.	M	/api/lab/schema	test_api.py::test_schema_explorer
FR-46	Every page that shows data states that the data is synthetic and that the safe-hour and carbon values are unverified.	M	SynthNote component	visible on every data page

4. Non-functional requirements

ID	Requirement	Target	How it is met	How it is checked
NFR-1	Data integrity	No invalid row can exist even if the application is bypassed	Keys, UNIQUE, CHECK and FK constraints plus triggers; procedures are the only write path	02_constraint_smoke_test.sql, 10_dml_examples.output.md
NFR-2	Concurrency	Zero double claims under simultaneous load	Row lock plus trigger plus unique generated column	13_race_demo.output.txt, race page
NFR-3	Query performance	Live feed, mess dashboard, audit timeline and fairness queries use an index, not a full scan	Sixteen indexes chosen per access path	12_explain_indexes.output.md, EXPLAIN page
NFR-4	Scalability	Plans stay index-based at 200,000 batch rows	Composite indexes on the filter and sort columns; spatial index for the radius pre-filter	12_explain_indexes.output.md (run at 200,000 rows)

ID	Requirement	Target	How it is met	How it is checked
NFR-5	Least privilege	No end-user role can read what its screens do not need (e-mails, password hashes, other sites' data)	Four roles plus two service roles, column-level grants, procedure-only writes	14_rbac_tests.output.md (66 of 66)
NFR-6	Auditability	Who changed what is always answerable; food-safety history cannot be rewritten	Audit triggers with USER() ; append-only hash-chained custody	admin page, custody page
NFR-7	Usability	A mess admin can post a batch in under a minute on a phone	One form, three required fields beyond the dropdowns; responsive layout	timed walk-through at the internal review, TO COLLECT
NFR-8	Accessibility	Keyboard reachable, visible focus, contrast-checked colours, reduced-motion fallback, no colour-only meaning	Design tokens with light and dark variants, :focus-visible, prefers-reduced-motion, labels on every control, chart palette validated for colour-vision deficiency	manual keyboard pass; palette validator output in the README
NFR-9	Honesty of numbers	No invented statistic appears anywhere	values_source column, TO VERIFY and TO COLLECT markers, synthetic-data note on every page	review of every document and page
NFR-10	Reproducibility	Anyone can rebuild the whole system from the repository	One setup.sql, deterministic seed generator, one-command local setup in the README	a clean rebuild before the review
NFR-11	Portability	Runs on a laptop with MySQL 8, Python 3 and Node	No cloud service; connection details come from environment variables	setup steps in the README
NFR-12	Maintainability	A new rule is added in one place	Business rules in SQL (procedures, triggers), policy in a table, no duplicated logic in the API	code review at the viva

4.1 Limitations stated honestly

- **No row-level security in MySQL.** "A shelter sees only its own rows" is enforced inside the procedures, which check `p_user_id`. The API connects per role and passes the authenticated user's id; end users never get database logins.
- **Session tokens are signed, not encrypted**, and the demo secret is regenerated at every restart. Good enough for a prototype, not for deployment.
- **Demo passwords are in the repository** (`demo1234`, and the `mb_*` database passwords in `08_roles_grants.sql`). They exist so an examiner can log in. The README says to change them before any real use.
- **No notifications leave the system.** `notification` rows are written, but nothing sends SMS or e-mail yet.
- **One campus, one city.** The 15 km service radius, 20 km/h average speed, 1.4 detour factor and 30 minute safety buffer are assumptions for a Vellore-scale pilot, listed in the README section 6.

5. Data requirements (summary)

The full specification is Deliverable 4 (`03_database_design.md`). In short:

- **24 tables.** `SITE` is a superclass with disjoint, total specialization into `MESS` and `SHELTER`, pinned by a generated discriminator column and a composite foreign key. `APP_USER` has a partial specialization into `VOLUNTEER`. `TRIP_STOP` is a weak entity of `PICKUP_TRIP`.
- **Normalized to 3NF, and BCNF where the keys allow it,** with the functional dependencies listed per table in the DDL comments and the justification in the design document, section 5.
- **Three deliberate, trigger-maintained denormalizations,** each justified: `surplus_batch.safe_until` (a frozen deadline snapshot), `shelter_day.reserved_kg` (a counter that a CHECK constraint can then police) and the score and distance snapshots on `claim` (the basis of a fairness audit, not derivable later).
- **Sixteen indexes,** each with the access path that justifies it, including two spatial indexes on SRID 4326 points for the radius pre-filter.
- **Reference data whose provenance is a column.** `food_category.values_source` records where each safe-hour and carbon figure came from; the seed values say `TO_VERIFY`.

6. Out of scope for this project

Mobile apps, payments, SMS or push delivery, routing via a maps API, multi-city tenancy, machine-learning forecasts, and anything that would need real personal data. Each would be a sensible next step after a field trial; none is needed to demonstrate the database design this course assesses.

7. Traceability summary

Deliverable	File	Covers
1. Problem Discovery	<code>01_problem_discovery.md</code>	problem, stakeholders, verified existing-solution study, fieldwork plan
2. Innovation Proposal	this document, section 1	I1 to I9, what is not claimed, impact metrics, TRL
3. Software Requirements	this document, sections 2 to 6	FR-1 to FR-46, NFR-1 to NFR-12, stakeholders, scope
4. Database Design	<code>03_database_design.md</code>	ER and EER diagrams, relational schema, FDs, normalization, indexing strategy
5. Application Prototype	<code>app/</code> , README section 8	FastAPI API and Next.js web app over the same database
6. Testing and Validation	<code>sql/*.output.md</code> , <code>app/backend/tests/test_api.py</code>	constraint, DML, query, index, concurrency, RBAC and API tests with real output
7. Impact and TRL	to be written after fieldwork	measured impact, justified TRL with evidence
8. Expo Presentation	to be written	demo script, poster outline, viva questions